



AEGIS
Anonymization & Exchange Gateway
for Imaging Studies

AEGIS Architecture

Anonymization & Exchange Gateway for Imaging Studies

Matthew L. Senjem, M.S. — February 18, 2026

SENDING SOURCES (EXTERNAL BROWSER + INTERNAL ENTERPRISE)

React Upload Portal (Web)

- DICOM file picker
- Client-side parsing (dcmjs)
- Tag anonymization preview (before/after)
- Optional email for notifications
- Project & profile selection
- Per-file progress + retry (3x backoff)

De-identification Engine (@aegis/client)

- DICOM PS3.15 Annex E Basic Profile
- 18 HIPAA Safe Harbor identifiers
- Per-project retained tags (anon profiles)
- StudyInstanceUID regeneration
- Runs in browser — PHI never leaves hospital

Batch Import CLI (Go)

- Server-side DICOM parsing (suyashkumar/dicom)
- Recursive directory scan
- Groups files by StudyInstanceUID
- Dry-run mode for preview
- Duplicate detection (safe re-run)

Key Principle

- PHI is stripped in the browser BEFORE data leaves the hospital network
- Only tag-de-identified DICOM is transmitted over HTTPS
- No software installation required at sending sites
- Batch import assumes pre-de-identified data



HTTPS (TLS 1.3) — De-identified DICOM only

Cloud Platform (GCP / AWS / Azure) — Secured Tenancy

Go API Backend (Cloud Run / ECS Fargate)

- Upload orchestration (init → file → complete)
- Study, project, institution management
- Routing rules engine (priority-ordered, all matching fire)
- DICOMweb proxy (QIDO-RS + WADO-RS for OHIF)
- Async dispatch to 6 Python sidecar services
- PostgreSQL migrations (goose)
- Export shares with time-limited tokens
- Centralized audit trail (every action logged)
- Email notifications (upload, approve, reject, digest)
- Health check: /healthz with DB + sidecar status

PostgreSQL 15 (Cloud SQL / RDS)

- Studies, upload sessions
- Projects, institutions
- Routing rules, destinations, routing log
- Anonymization profiles
- Protocol templates (JSONB rules)
- Admin users (RBAC)
- Digest subscriptions
- Export shares
- Audit entries (actor, event, detail)
- 16 migrations

Object Storage (GCS / S3 / Local)

- Pluggable Storage interface (8 methods)
- Presigned upload/download URLs
- dicom/raw/{studyUID}/ — pre-defacing
- dicom/clean/{studyUID}/ — post-defacing
- bids/{studyUID}/ — BIDS output
- KMS encryption at rest
- Lifecycle rules for staging cleanup

PYTHON SIDECAR SERVICES (FASTAPI, ASYNC DISPATCH FROM GO API)

Defacing Service

- 4 backends: DeepDefacer, mri_deface, mri_reface, nibabel
- DICOM → NIFTI → deface → DICOM
- Head imaging only (auto-routed)
- Side-by-side review in admin

PHI Detection

- Tesseract OCR on pixel data
- Detects burned-in text overlays
- Confidence threshold filtering
- Findings → audit trail

QC Service

- 5 checks: integrity, consistency, SNR, coverage, gaps
- pydicom + numpy
- pass / warn / fail status
- Quality issues → audit trail

BIDS Conversion

- dcm2niix backend
- BIDS directory structure
- Series → datatype classification
- Zip download endpoint

Classification

- Heuristic DICOM tag analysis
- Modality + body part detection
- Re-evaluates routing rules
- SOP Class / description matching

Protocol Compliance

- Classic + Enhanced DICOM
- Per-scanner template matching
- TR, TE, flip angle, resolution
- numeric / exact / range rules

React Admin Dashboard (Web)

- Study management: filter, search, paginate (50/page), approve/reject
- OHIF Viewer integration (embedded iframe + new tab)
- Side-by-side defacing review (raw vs clean data source)
- Pipeline badges: PHI scan, QC, BIDS, Classification, Protocol
- 9 tabs: Studies, Audit Log, Routing, Institutions, Profiles, Protocol Templates, Notifications, Projects, Users
- RBAC: admin (full) / viewer (read-only), Users tab hidden for viewers

Authentication & Security

- Multi-provider auth middleware (auto-detect)
- GCP: Identity-Aware Proxy (IAP)
- AWS: ALB + Cognito (JWT)
- Azure: AD Easy Auth
- Dev: auto-auth via DEV_USER_EMAIL
- RBAC: RequireAuth / RequireRole("admin")
- CORS, rate limiting, TLS 1.3

Routing Rules Engine

- Conditions: project, modality, body part, source
- Actions: require_defacing, require_phi_scan, require_qc_check, require_bids_conversion, require_classification, require_protocol_check
- auto_approve, reject, route_to destination
- Priority-ordered, all matching rules fire
- Re-evaluation after classification

INFRASTRUCTURE AS CODE (TERRAFORM)

GCP Module (terraform/infra/)

- Cloud Run (API + sidecars)
- Cloud SQL PostgreSQL 15
- GCS bucket + KMS
- Cloud Armor WAF
- Identity-Aware Proxy
- VPC + Private Service Connect

AWS Module (terraform/aws/)

- ECS Fargate cluster
- RDS PostgreSQL 15
- S3 bucket + KMS
- Application Load Balancer
- Cognito user pool
- VPC (2 AZ, public + private subnets)
- ECR repositories (7 services)
- SNS + SQS event bus

Local Development (Docker Compose)

- PostgreSQL + Mailpit + OHIF Viewer
- Go API + 6 Python sidecars
- Shared aegis-data volume
- Health checks on all services
- Makefile: up, down, clean, build, check, logs
- GitHub Actions CI (Go + Python + TS + Docker)

Key Open-Source Dependencies

- **Go:** suyashkumar/dicom, goose, pgx
- **DICOM:** dcmjs, dicomParser, dcm2niix
- **Defacing:** DeepDefacer, mri_deface, nibabel, pydicom
- **PHI:** Tesseract OCR, Pillow
- **Viewer:** OHIF Viewer v3
- **Infra:** Terraform, Docker, GitHub Actions

IMPLEMENTATION PHASES

Phase 1: Foundation Code Complete

Terraform (GCP + AWS), Go API, Upload Portal, Admin Dashboard, OHIF Viewer

Phase 2: Defacing Pipeline Code Complete

mri_deface, side-by-side review, email notifications (upload/approve/reject)

Phase 3: Operations Code Complete

Routing rules, institutions, anon profiles, audit log, admin users, RBAC, auth middleware

Phase 4: Advanced Processing Code Complete

PHI detection, QC, BIDS, classification, protocol compliance, batch import, multi-cloud

Next: Test Deployment Planned

Deploy to GCP/AWS, end-to-end test with TCIA data, performance benchmarking

Phase 5: Enterprise Future

DIMSE receive, HL7/FHIR, tag standardization, reversible de-id, multi-tenant SaaS